SkillGraph: 4-Member Team Codebase Division & Oral Exam Defense Guide

Complete File Ownership, Technical Explanations, Internal Mechanics, and Teacher Q&A Preparation

> **Objective:** Every team member has an equal, specialized ownership of Frontend (HTML/CSS/JS), Backend (Routers/Services/Schemas), Data/Models, and Automated Tests. When the teacher asks any member to explain a file, that member can explain the code line-by-line, its architectural purpose, and defend it with confidence.

1. Master Team Division Matrix

Member	Role & Specialization	Frontend Files	Backend Files (API & Services)	Database & Tests
Member 1 (You / Lead)	**Platform Architecture, Auth & Resume Ingestion**	`index.html`, `dashboard.html`, `app.js`, `dashboard.js`, `dashboard.css`	`main.py`, `config.py`, `auth.py` (API/Svc/Sch), `resume.py` (API/Sch), `resume_parser.py`	`database.py`, `models/user.py`, `resume.py`, `test_auth.py`, `test_resume.py`, `test_database.py`, `test_health.py`
Member 2	**Resume Quality Advisor & Coding Signal Harvesters**	`resume-review.html`, `coding.html`, `resume-review.js`, `coding.js`	`profiles.py` (API/Sch), `coding_profiles.py`, `resume_advisor.py` (API/Svc/Sch)	`test_resume_advisor.py`, `test_coding_profiles.py`
Member 3	**Skill Taxonomy, Role Gap Analysis & Job Market Analytics**	`analytics.html`, `roles.html`, `analytics.js`, `roles.js`	`skills.py`, `roles.py`, `job_descriptions.py` (API/Svc/Sch), `skill_extractor.py`, `skill_analyzer.py`, `role_discovery.py`, `market_insights.py`, `job_market_analytics.py`	`models/job_description.py`, `analysis.py`, `role_profiles.json`, `skill_aliases.json`, `verified_market_data.json`, `test_skills.py`, `test_role_discovery.py`, `test_job_description.py`, `test_job_market_analytics.py`
Member 4	**Roadmap Generation, Design System & DevOps Infrastructure**	`roadmap.html`, `roadmap.js`, `variables.css`, `style.css`, `components.css`, `roadmap.css`, `config.js`, `api.js`, SVG icons	`roadmap.py` (API/Svc/Sch), `roadmap_generator.py`	`models/roadmap.py`, `progress.py`, `Dockerfile`, `.dockerignore`, `run.bat`, `test_roadmap.py`, `test_frontend_integration.py`, `test_workflow.py`

MEMBER 1: Platform Architecture, Auth & Resume Parsing

Primary Focus: Application bootstrap, ASGI security/CORS, user authentication lifecycle, file byte security, and raw resume extraction.

Files to Read & Master:

backend/app/main.py & backend/app/config.py

What it does: The central FastAPI application factory. `config.py` uses `pydantic_settings.BaseSettings` to load environment variables (SECRET_KEY, CORS_ORIGINS, DATABASE_URL) with safe fallbacks. `main.py` configures CORS middleware, declares the async `lifespan()` context manager that initializes database tables, registers all 7 API routers, and mounts the `/frontend` directory as static files.

backend/app/database.py, models/user.py, models/resume.py

What it does: `database.py` configures SQLAlchemy's `create_engine` and `sessionmaker` with SQLite connection settings (disabling same-thread checks for async compatibility). `models/user.py` defines the User table with hashed passwords.

`models/resume.py` defines the Resume table with foreign key linking to User, storing raw extracted text, parsed skill JSON, and timestamps.

backend/app/services/resume_parser.py & api/resume.py

What it does: Implements safe file ingestion. Checks *magic bytes*: PDF files must start with `%PDF-` and DOCX with `PK``. If someone renames an .exe or malware file to .pdf, it is rejected immediately. It extracts plain text using `pymupdf` for PDFs and `python-docx` for Word documents, normalizes whitespace, and extracts detected skills.

frontend/index.html, dashboard.html, js/app.js, js/dashboard.js

What it does: `index.html` is the landing page with mobile drawer toggle and CTA navigation. `dashboard.html` provides the drag-and-drop resume upload zone, file reader, readiness hero scorecard, and radar chart. `dashboard.js` listens for file upload events, calls `API.resume.upload()`, and renders skill tags dynamically.

Expected Teacher Questions for Member 1:
Q: How do you prevent users from uploading malicious executable files disguised as PDFs?
Answer: In `resume_parser.py`, we don't rely on the file extension string. We inspect the first few bytes (the "magic bytes"). Valid PDFs start strictly with `b'%PDF-'`. Valid Word DOCX files are zip archives starting with `b'PK``'`. If magic bytes don't match, we raise an `HTTP 400 Bad Request`.
Q: How does authentication work without sessions?
Answer: In `services/auth.py`, we hash passwords using `passlib` with bcrypt. Upon successful login, we generate an HMAC-SHA256 JSON Web Token (JWT) containing the user ID and expiration time. The frontend stores this token and sends it in the `Authorization: Bearer <token>` header.

MEMBER 2: Resume Quality Advisor & Coding Signal Harvesters

Primary Focus: AI candidate evaluation, Google XYZ bullet rewriting, GitHub REST API, LeetCode GraphQL, and DSA Interview Readiness.

Files to Read & Master:

backend/app/services/resume_advisor.py & schemas/resume_advisor.py

What it does: Audits resume text across 4 weighted dimensions:

1. **Metric Quantification (0-25):** Identifies metrics ($ values, %, numbers, scale) representing impact.
2. **Action Verb Strength (0-25):** Checks for power verbs (*architected, spearheaded, accelerated*) vs weak passive verbs (*worked on, helped, responsible for*).
3. **Keyword Density (0-25):** Calculates density of canonical tech skills.
4. **Fluff Penalty (0-25):** Penalizes non-technical buzzwords (*go-getter, guru, hard worker*).

Generates concrete *Things to Add*, *Things to Remove*, and optimizes weak bullets using Google's XYZ formula: *"Accomplished [X] as measured by [Y], by doing [Z]"*.

backend/app/services/coding_profiles.py & api/profiles.py

What it does: Uses asynchronous `httpx.AsyncClient`:

- **GitHub:** Queries `https://api.github.com/users/{handle}` and `/repos` to fetch public repo counts, stargazers, and primary languages.
- **LeetCode:** Queries LeetCode's public GraphQL endpoint (`https://leetcode.com/graphql`) with the `matchedUser` query to retrieve solved problem counts categorized into Easy, Medium, and Hard.
- **DSA Readiness Index:** Computes a 0–100 score: `Readiness = (Easy * 0.15 + Medium * 0.55 + Hard * 0.30) / TargetThreshold * 70 + GitHubActivity * 30`.

frontend/resume-review.html, coding.html, js/resume-review.js, js/coding.js

What it does: `resume-review.html` renders the 0-100 gauge, 4 score breakdown progress bars, addition/removal pill tags, and before/after bullet comparison cards. `coding.html` renders dual profile inputs, SVG readiness meters, LeetCode Easy/Med/Hard breakdown bars, and interview recommendations.

Expected Teacher Questions for Member 2:
Q: How do you extract LeetCode data if there is no official REST API?
Answer: In `coding_profiles.py`, we send an asynchronous GraphQL `POST` request to `https://leetcode.com/graphql` with the query `getUserProfile($username)`. It queries LeetCode's public schema to return `submitStats.acSubmissionNum`, giving

exact counts of solved Easy, Medium, and Hard problems without requiring user passwords.
Q: Explain the Google XYZ resume bullet formula.
Answer: In `resume_advisor.py`, weak bullets like *"Worked on database speed"* lack metrics and context. We detect the missing metric and transform it to: *"Accomplished 40% reduction in database query latency [X] as measured by Datadog APM [Y], by implementing Redis caching and compound indexing [Z]"*.

MEMBER 3: Skill Taxonomy, Gap Analysis & Job Market Analytics

Primary Focus: Skill ontology matching, role requirement benchmarks, traditional vs emerging roles, and cross-role co-occurrence analytics.

Files to Read & Master:

backend/app/services/skill_extractor.py & skill_analyzer.py

What it does: `skill_extractor.py` normalizes text and matches against canonical skills using word-boundary regular expressions (e.g. `r'(?i)\bReact\b'`). It uses `skill_aliases.json` so that "Postgres" maps to "PostgreSQL" and "K8s" maps to "Kubernetes". `skill_analyzer.py` calculates match percentages against target roles: `Coverage = (matched_core * 2 + matched_secondary) / (total_core * 2 + total_secondary)`.

backend/app/services/role_discovery.py & job_market_analytics.py

What it does: `role_discovery.py` manages 10 traditional and emerging roles stored in `role_profiles.json` (including *Harness Engineer*, *Forward Deployed Engineer*, and *Agentic AI Engineer*). `job_market_analytics.py` aggregates skill frequencies across all roles and computes co-occurrence pairs (e.g. Python + FastAPI appears in 60% of backend roles).

frontend/analytics.html, roles.html, js/analytics.js, js/roles.js

What it does: `analytics.html` renders interactive market demand bar charts, high-signal skill pairings, and empirical role benchmark tables. `roles.html` renders the 10 role cards, skill category filters (Languages, Frameworks, Cloud, ML), and the interactive dual-role comparison tool.

Expected Teacher Questions for Member 3:
Q: How do you prevent false-positive skill matches (e.g., matching the skill 'Go' inside the word 'Good')?
Answer: In `skill_extractor.py`, we compile regexes using word boundary anchors: `r'(?i)\b' + re.escape(alias) + r'\b'`. For short 1-2 letter acronyms like 'C' or 'R', we require strict uppercase tokenization or contextual word boundaries so plain English words never trigger false skill matches.
Q: What is a Harness Engineer and why is it included?
Answer: Harness Engineers specialize in automated testing harnesses, CI/CD orchestration, load generation frameworks, and chaos engineering. They are crucial in modern platform engineering to ensure software reliability before deployment.

MEMBER 4: Roadmap Engine, Design System & DevOps Infrastructure

Primary Focus: DAG prerequisite resolution, 6-week milestone scheduling, Paper & Ink OKLCH design system, unified API client, and Docker containerization.

Files to Read & Master:

backend/app/services/roadmap_generator.py & api/roadmap.py

What it does: Takes candidate skill gaps and constructs a personalized 6-week learning roadmap. Uses a Directed Acyclic Graph (DAG) topological dependency sort:

- **Weeks 1–2 (Foundation):** Core language prerequisites and fundamental concepts.
- **Weeks 3–4 (Core Frameworks):** Application frameworks, databases, and API development.
- **Weeks 5–6 (System Integration & Capstone):** Production deployment, CI/CD, and mock interview questions.

Stores weekly tasks in SQLite and tracks completion checkboxes via `models/progress.py`.

frontend/css/variables.css, style.css, components.css, roadmap.css

What it does: The *Paper & Ink* design system. Uses modern CSS OKLCH color spaces (`--background: oklch(0.98 0.01 85)` for warm parchment and `--foreground: oklch(0.20 0.02 55)` for deep ink). Fully responsive layout with CSS Grid and Flexbox, custom typography hierarchy (Fraunces serif headings, Inter body, JetBrains Mono code).

frontend/js/api.js, config.js, roadmap.js & Dockerfile

What it does: `api.js` is the centralized API client wrapping `fetch()` with automatic token injection and error handling. `Dockerfile` is a multi-stage production build using `python:3.13-slim`, exposing port 8000 with volume mount `/app/data` for persistent SQLite storage.

> **Expected Teacher Questions for Member 4:**
> **Q: How does the roadmap generator decide the sequence of skills across the 6 weeks?**
> *Answer:* In `roadmap_generator.py`, skills have prerequisite relationships. You cannot learn FastAPI before Python, or Kubernetes before Docker. We perform topological dependency sorting to schedule prerequisites into Weeks 1–2, core frameworks into Weeks 3–4, and production capstone projects into Weeks 5–6.
> **Q: Why did you build the frontend in Vanilla JS and CSS instead of React or Tailwind?**
> *Answer:* To achieve zero runtime overhead, instant page loads, and complete independence from complex build tooling (no Node.js/npm dependencies). Native HTML5/CSS3 with OKLCH design variables provides modern, high-fidelity design without framework bloat.

5. Oral Defense Strategy & Teacher Presentation Walkthrough

1. **Start with Live System Proof:** Member 1 boots the server using `run.bat` and displays `http://127.0.0.1:8000/docs` and `/api/health`.
2. **Pass the Baton:**
   - **Member 1:** Demonstrates landing page (`index.html`) and uploads a sample resume on `dashboard.html`.
   - **Member 2:** Shows the `resume-review.html` audit score and explains the LeetCode GraphQL fetcher on `coding.html`.
   - **Member 3:** Shows the market trend analytics on `analytics.html` and explains the role comparison on `roles.html`.
   - **Member 4:** Demonstrates the generated 6-week plan on `roadmap.html`, checks off a completed task, and explains the Docker architecture.
3. **Conclude with Test Suite:** Run `python -m pytest tests -q` showing all **172 passed tests in ~8 seconds**.